

MULTITHREADED PROGRAMMING-II



Need for Synchronization

- **Synchronization** is required in computing systems to **coordinate access to shared resources** when multiple processes or threads execute concurrently.
- Synchronization is the mechanism used to control the execution order of concurrent processes or threads so that shared data is accessed in a safe and consistent manner.

■ **Key Problems Without Synchronization**

■ **1. Race Condition**

- Occurs when:
- Two or more processes access shared data
- Final result depends on execution order

■ **Example:**

- Two threads increment the same variable
- Expected result = 2
- Actual result may be 1

■ **2. Data Inconsistency**

- Partial updates lead to incorrect data
- Database records may become corrupted

■ **3. Lost Updates**

- One process overwrites changes made by another

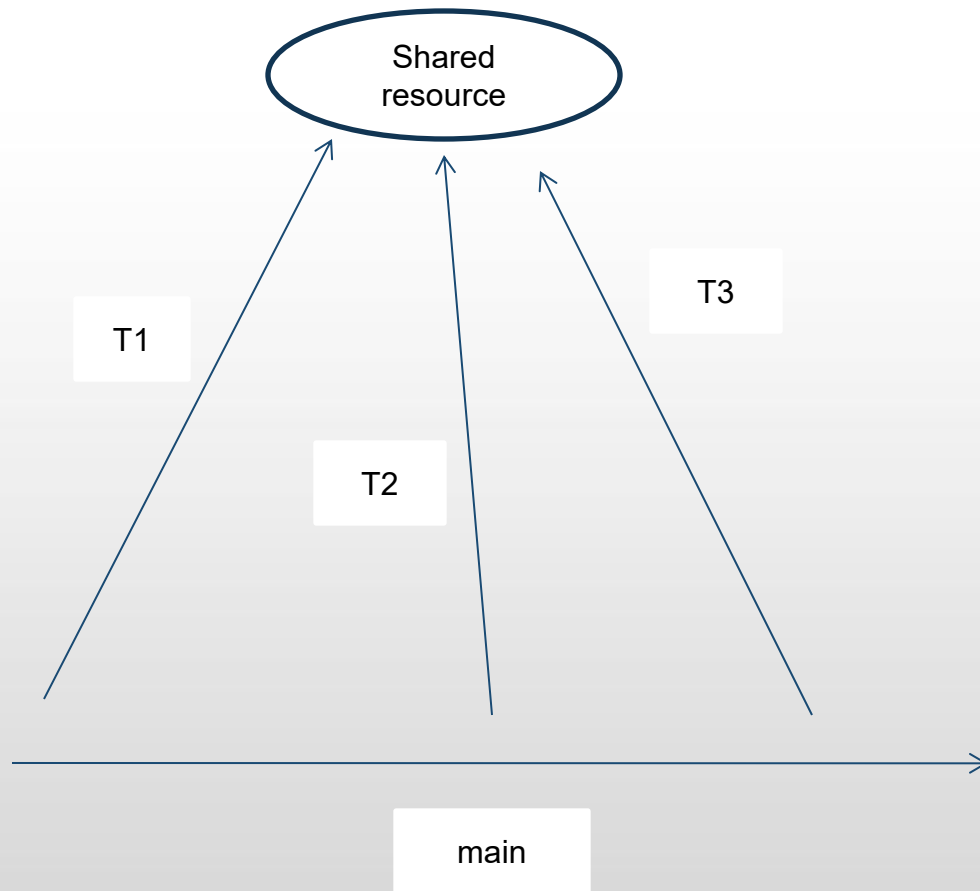
■ **4. Critical Section Problem**

- A **critical section** is a part of the program where shared resources are accessed.
- Without synchronization:
- Multiple processes may enter the critical section simultaneously

- **1. synchronized Keyword (Core Mechanism)**
- Java uses the **synchronized** keyword to allow **only one thread at a time** to access a critical section.
- **a) Synchronized Method**
- `synchronized void update() {`
- `// critical section`
- `}`
- Locks the **object's monitor**
- Other threads must wait until the lock is released
- **b) Synchronized Block**
- `synchronized(lockObject) {`
- `// critical section`
- `}`
- Finer control than synchronized methods
- Improves performance by locking only required code

Synchronization

- When two or more threads need concurrent access to a shared data resource, they need to take care to only access the data one at a time.
- For example, one thread may try to read a record from a file while another is still writing to the same file. Depending on the situation, we may get strange results.
- Java enables us to overcome this problem using a technique known as synchronization.
- Keyword synchronized helps to solve such problem by keeping a watch on such locations.



```
synchronized void update()
```

```
{
```

```
}
```

- When we declare a method synchronized, java creates a “monitor” and hands it over to the thread that calls the method first time.
- As long as the thread holds the monitor, no other thread can enter the synchronized section of the code.
- A monitor is like a key and the thread that holds the key can only open the lock.

It is also possible to mark a block of code as synchronized as follows.

```
synchronized(lock-object)
{
    // code here
}
```

Whenever a thread has completed its work of using synchronized method(or block of code), it will handover the monitor to the next thread that is ready to use the same resources.


```
class Callme
{
    void call(String msg)
    {
        System.out.print("[ " + msg);
        try
        {
            Thread.sleep(1000);
        }
        catch(InterruptedException e)
        {
            System.out.println("Interrupted");
        }
        System.out.print("] " );
    }
}
```

```
class Caller extends Thread
{
    String msg;
    Callme obj;
    public Caller(Callme targ, String s)
    {
        msg = s;
        obj=targ;
    }
    public void run()
    {
        obj.call(msg);
    }
}
```

```
class Synch3
```

```
{    public static void main(String args[])
```

```
{        Callme target = new Callme();
```

```
        Caller c1= new Caller(target, " manipal ");
```

```
        c1.start();
```

```
        Caller c2= new Caller(target, "institute");
```

```
        c2.start();
```

```
        Caller c3= new Caller(target, "of");
```

```
        c3start();
```

```
        new Caller(target, " technology ").start();
```

```
    }
```

```
}
```

OUTPUT

[manipal [institute [of [technology]]]]

```
class Callme
{
    synchronized void call(String msg)
    {
        System.out.print "[" + msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            System.out.println("Interrupted");
        }
        System.out.print "]" );
    }
}
```

OUTPUT

[manipal] [institute] [of] [technology]

- Previous program may be written by removing Synchronized keyword and updating run method as follows

Marking block of code as synchronized

```
public void run()
{
    synchronized(obj)
    {
        obj.call(msg);
    }
}
```


Interthread Communication

Methods must be called from inside of synchronized method.

All three are final methods. Defined in Object class

1. **wait()** tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls **notify()**.
final void wait() throws InterruptedException
2. **notify()** wakes up the first thread that called **wait()** on the same object.
final void notify()
3. **notifyAll()** wakes up all the threads that called **wait()** on the same object. The highest priority thread will run first.
final void notifyAll()

// An incorrect implementation of a producer and consumer.

```
class Q
{
    int n;
    synchronized int get()
    {
        System.out.println("Got: " + n);
        return n;
    }

    synchronized void put(int n)
    {
        this.n = n;
        System.out.println("Put: " + n);
    }
}
```

```
class Producer implements Runnable
{
    Q    q;
    Producer(Q    q)
    {
        this.q = q;
        new Thread(this, "Producer").start();
    }

    public void run()
    {
        int i = 0;
        while(true)
        {
            q.put(i++);
        }
    }
}
```

```
class Consumer implements Runnable
{
    Q q;
    Consumer(Q q)
    {
        this.q = q;
        new Thread(this, "Consumer").start();
    }
    public void run()
    {
        while(true)
        {
            q.get();
        }
    }
}
```

```
class PC
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Q q = new Q();
```

```
        new Producer(q);
```

```
        new Consumer(q);
```

```
        System.out.println("Press Control-C to stop.");
```

```
    }
```

```
}
```

Output

Put: 1

Got: 1

Got: 1

Got: 1

Got: 1

Got: 1

Put: 2

Put: 3

Put: 4

Put: 5

Put: 6

Put: 7

Got: 7

Correct implementation using wait() and notify()

// A correct implementation .

```
class Q
{   int n;
    boolean valueSet = false;
    synchronized int get()
    {   if(!valueSet)
        try
        {   wait();   }
        catch(InterruptedException e)
        {   System.out.println("Exception ");   }

        System.out.println("Got: " + n);
        valueSet = false;
        notify();
        return n;
    }
```

```
synchronized void put(int n)
{   if(valueSet)
    try
    {   wait();   }
    catch(InterruptedException e)
    {   System.out.println("Exception");   }

    this.n = n;
    valueSet = true;
    System.out.println("Put: " + n);
    notify();
}}
```

class Producer implements Runnable

```
{  
    Q q;  
    Producer(Q q)  
    {  
        this.q = q;  
        new Thread(this, "Producer").start();  
    }  
    public void run()  
    {  
        int i = 0;  
        while(true)  
        {  
            q.put(i++);  
        }  
    }  
}
```


class Consumer implements Runnable

```
{  
    Q q;  
    Consumer(Q q)  
    {  
        this.q = q;  
        new Thread(this, "Consumer").start();  
    }  
    public void run()  
    {  
        while(true)  
        {  
            q.get();  
        }  
    }  
}
```

```
class PCFixed
{
    public static void main(String args[])
    {
        Q q = new Q();
        new Producer(q);
        new Consumer(q);
        System.out.println("Press Control-C to stop.");
    }
}
```

Output showing synchronous behaviour

Put: 1

Got: 1

Put: 2

Got: 2

Put: 3

Got: 3

Put: 4

Got: 4

Put: 5

Got: 5

Deadlock

- **Deadlock** occurs when two threads have a circular dependency on a pair of synchronized objects.
- **For ex:**
- Suppose one thread enters the monitor on object X and another thread enters the monitor on object Y.
- If the thread in X tries to call any synchronized method on Y, it will block as expected.
- If the thread in Y, in turn, tries to call any synchronized method on X, the thread waits forever, because to access X, it would have to release its own lock on Y so that the first thread could complete.

```
■ class DeadlockExample {  
  
■     static final Object lock1 = new Object(); //Common base class Object  
                                                //defined in java.lang  
  
■     static final Object lock2 = new Object();  
  
■     public static void main(String[] args) {  
  
■         Thread t1 = new Thread(new Task1());  
■         Thread t2 = new Thread(new Task2());  
  
■         t1.start();  
■         t2.start();  
  
■     }  
  
■ }
```

- class Task1 implements Runnable {
- public void run() {
- synchronized (DeadlockExample.lock1) {
- System.out.println("Thread 1: Holding lock1");
- try {
- Thread.sleep(100);
- } catch (InterruptedException e) {
- e.printStackTrace();
- }
- synchronized (DeadlockExample.lock2) {
- System.out.println("Thread 1: Holding lock2");
- }
- }
- }

- class Task2 implements Runnable {
- public void run() {
- synchronized (DeadlockExample.lock2) {
- System.out.println("Thread 2: Holding lock2");
- try {
- Thread.sleep(100);
- } catch (InterruptedException e) {
- e.printStackTrace();
- }
- synchronized (DeadlockExample.lock1) {
- System.out.println("Thread 2: Holding lock1");
- }
- }
- }